Investigación sobre Named Pipes en C#

1-Investigar sobre named pipelines en C#

¿Qué son las Named Pipes?

Una **Named Pipe** (tubería con nombre) es un canal de comunicación que permite que dos procesos independientes intercambien datos. Un proceso, el **servidor**, crea la tubería y le da un nombre. Otro proceso, el **cliente**, se conecta a ella usando ese mismo nombre. Es lo que se conoce como **IPC** (Inter-Process Communication, comunicación entre procesos).

En C# se utilizan las clases del espacio de nombres System.IO.Pipes. Una vez que cliente y servidor están conectados, se leen y escriben datos igual que en cualquier otro Stream (como un archivo), con StreamReader y StreamWriter.

Y nuestro proyecto tiene dos aplicaciones de escritorio independientes (dos procesos distintos) que envían información.

Características principales

- **Tienen nombre:** el cliente localiza la tubería por su nombre (en nuestro caso EscapeRoomsPipe). Si el nombre no coincide, no hay conexión.
- **Comunican procesos independientes:** cada aplicación tiene su propio proceso y su propia memoria; no comparten variables, por eso necesitan un canal para enviarse datos.
- **Pueden ser unidireccionales o bidireccionales:** con PipeDirection.In, Out o InOut.
- **Funcionan en la misma máquina o por red** (en Windows). En nuestro caso, las dos aplicaciones se ejecutan en el mismo ordenador.
- **Son bloqueantes por defecto:** esperar una conexión o leer datos detiene el hilo que lo hace. Esto es importante para la interfaz gráfica (ver apartado 4.4).

Clases y métodos principales

Elemento	Para qué sirve
NamedPipeServerStream	Crea la tubería en el lado del servidor (Aplicación 2).
NamedPipeClientStream	Se conecta a la tubería desde el cliente (Aplicación 1).
PipeDirection.InOut	Permite enviar y recibir por la misma conexión.
WaitForConnection() / WaitForConnectionAsync()	El servidor espera hasta que un cliente se conecta.
Connect(timeout)	El cliente se conecta, esperando como máximo el tiempo indicado (milisegundos).
StreamWriter / StreamReader	Escribir y leer texto sobre la tubería (WriteLine / ReadLine).

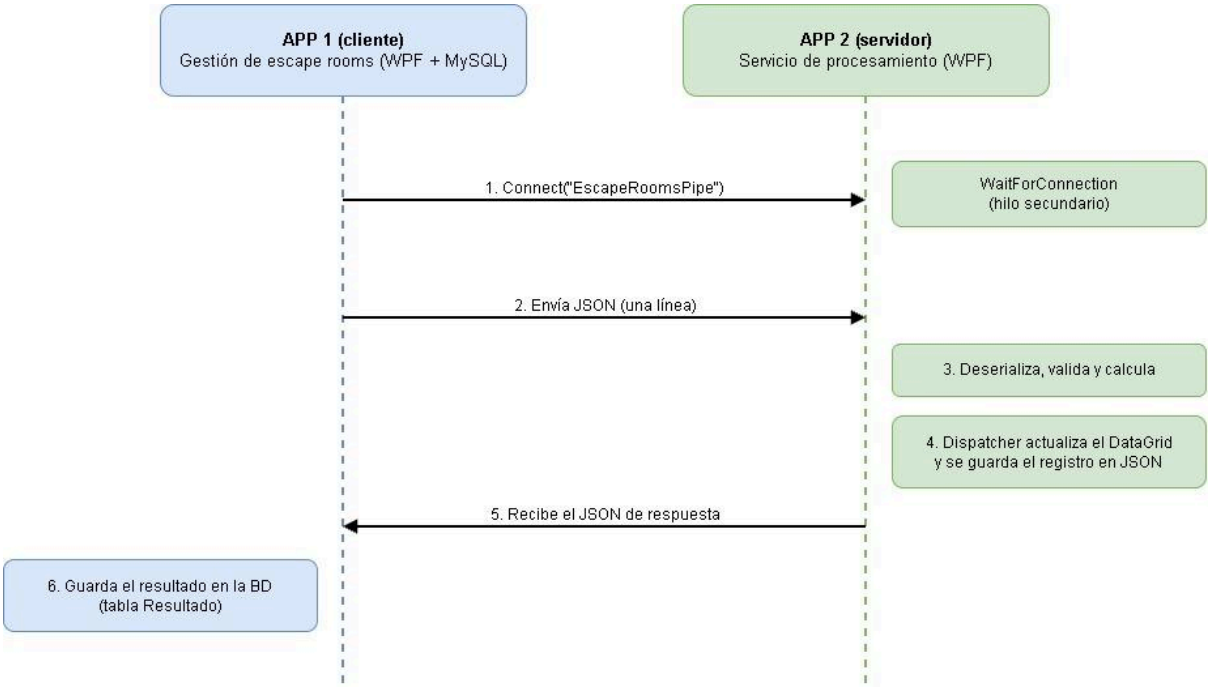
JsonSerializer.Serialize() / Deserialize<T>()	Convertir un objeto a JSON y viceversa (System.Text.Json).

Cómo funciona paso a paso

1. El servidor crea la tubería con un nombre **(NamedPipeServerStream).**
2. El servidor se queda esperando con **WaitForConnection.**
3. El cliente se conecta con **Connect**, usando el mismo nombre.
4. El cliente escribe un mensaje y el servidor lo lee.
5. El servidor procesa el mensaje y escribe la respuesta; el cliente la lee.
6. Se cierran los streams y la conexión termina. El servidor crea una nueva tubería y vuelve a esperar.

2-Explicar cómo lo vais a usar en vuestro proyecto

Esquema general:



Reparto:

- **La Aplicación 2** es el servidor (NamedPipeServerStream): recibe la solicitud, la válida, calcula la puntuación y devuelve el resultado.
- **La Aplicación 1** es el cliente (NamedPipeClientStream): envía la solicitud de una partida y guarda la respuesta en la tabla Resultado de la base de datos.
- Los dos usan exactamente el mismo nombre de pipe: **EscapeRoomsPipe .**
- **La App 2** debe estar abierta antes de que la App 1 intente conectarse.

2.3. Por qué se usa un hilo secundario y el Dispatcher

WaitForConnection bloquea el hilo que lo ejecuta hasta que alguien se conecta. Si lo ejecutáramos en el hilo de la interfaz (el hilo de la UI), la ventana de la App 2 se congelaría y no respondería.

Por eso la escucha se ejecuta en una tarea en segundo plano (Task.Run). Pero WPF solo permite modificar los controles desde el hilo de la UI, así que cuando llega una petición y hay que añadir una fila al DataGrid, se usa el Dispatcher:

```
await Application.Current.Dispatcher.InvokeAsync(() =>
{
    peticiones.Add(registro);    // ObservableCollection<PeticionRegistro>
});
```

La colección ObservableCollection está enlazada al DataGrid mediante *data binding*, así que al añadir un elemento la tabla se actualiza sola.

5. Ejemplos de JSON

Datos enviados (App 1 → App 2)

```
{
  "idSolicitud": "SOL-2026-001",
  "reserva": {
    "idReserva": 1,
    "sala": "La Mansión Maldita",
    "numJugadores": 4,
    "valida": true
  },
  "partida": {
    "tiempoSegundos": 3240,
    "duracionMaxMin": 60,
    "pistasUsadas": 2
  }
```

}

Datos recibidos (App 2 → App 1)

{
  "idSolicitud": "SOL-2026-001",
  "estado": "Procesada correctamente",
  "puntuacion": 780,
  "clasificacion": "Excelente"
}

Respuesta con error de validación

{
  "idSolicitud": "SOL-2026-009",
  "estado": "Error de validación",
  "campoError": "partida.tiempoSegundos",
  "detalle": "El tiempo supera la duración máxima de la sala",
  "puntuacion": null,
  "clasificacion": null
}